

Roguelike Game

OOP with Java 1 - Exam Project

Group 03: Hansmar Poulsen

Student id: 2004.024

May 30, 2026

1 Introduction

The objective of this project was to design and implement a roguelike console-based game built on the principles of object-oriented programming. The game is a small dungeon crawler on a turn-based grid, where ASCII characters represent walls, monsters, items, and the player. Each input advances the world by one step: enemies move, fuses tick down, traps fire. The player wins by reaching the treasure on the final level and loses by running out of health. Output is rendered in a terminal using ANSI escape codes for color. The goal of the project was to use Java's object-oriented features - abstraction, encapsulation, inheritance and polymorphism - so that adding a new monster or item would mean writing one new class and changing as little of the existing code as possible.

I approached the project by building the smallest playable version first; a player moving on a grid with walls and then expanding from there. The early effort went into getting the **Entity** right, because every later feature would extend monsters, traps, the bomb, the staircase and so on. This report describes that design, the data-driven level format that emerged from it, and the tests that verify its core behaviors.

Contents

1	Introduction	1
2	Design overview	3
2.1	Class structure and inheritance hierarchy	3
2.2	Encapsulation	4
2.3	Polymorphism in practice and key design decisions	5
2.4	Testing strategy	6
3	Limitations and future work	7
4	Reflection	8

2 Design overview

2.1 Class structure and inheritance hierarchy

Almost everything visible in the dungeon: walls, water, fire, traps, items on the floor, monsters, the staircase, the treasure, the player and lit bombs, inherits from the abstract base class **Entity**. **Entity** defines three things: a position, a rendering contract, and an interaction contract. Subclasses fill in the specifics.

```
public abstract class Entity {
    protected int x, y;
    private boolean consumed;

    protected abstract char getSymbol();
    protected abstract Color getFg();
    protected abstract Color getBg();

    public final void renderOn(Canvas canvas) {
        canvas.set(x, y, getSymbol(), getFg(), getBg());
    }

    public boolean isOccupying() { return true; }
    public abstract String interactWith(Player player);
    public String tick(Dungeon dungeon, Player player) { return ""; }
}
```

The rendering setup uses Java's **final** keyword to lock in how cells are drawn: **renderOn** is **final**, so no subclass can override it. Subclasses only supply the three pieces of data (**getSymbol**, **getFg**, **getBg**) and the parent does the actual canvas write. This means I cannot accidentally introduce two different rendering paths; the rendering happens in exactly one place.

interactWith is abstract: every entity must declare what happens when the player bumps it. Walls return an empty string; treasure sets a flag on the player; fire damages the player and decrements a counter. By forcing every subclass to write a method, even if the method does nothing, I avoid the failure mode where a new entity type silently inherits some default behaviour I forgot to override.

tick, by contrast, has an empty default. Most entities don't act on their own turn: walls don't move, water doesn't attack. Only enemies, the bomb, and the arrow trap need to override it. Making **tick** concrete-with-default means I don't pollute every wall subclass with an empty **tick** method just to satisfy the compiler. **interactWith** applies the same principle in reverse: when there is meaningful default behaviour for most subclasses (no-op tick), inherit it; when there isn't (interaction differs every time), force the override.

Two intermediate classes sit between **Entity** and the leaf classes. **LivingEntity** adds HP, healing, damage, and a death check. It is extended by both **Player** and **Enemy**. **Enemy** itself is abstract and adds a uniform combat exchange (player walks into the enemy, both deal damage to each other, enemy may die) plus two shared movement helpers, **isAdjacentTo** for adjacency checks and **stepToward** for greedy single-tile pursuit. The four concrete enemy types **Zombie**, **Ghoul**, **CaveDog** and **Slime** extend **Enemy** and differ only in their stats, visuals, and **tick** logic. **Zombie** moves one tile per turn; **Ghoul** and **CaveDog** both move two; **Slime** acts only every other turn, which required introducing per-entity state that persists across ticks. Each enemy expresses a genuinely different behavior through its **tick** override.

The full hierarchy looks like this:

```
Entity
|-- LivingEntity (abstract)
|   |-- Player
|   |-- Enemy (abstract)
|       |-- Zombie
|       |-- Ghoul
|       |-- CaveDog
|       |-- Slime
|   '-- DestructibleWall
|-- IndestructibleWall
|-- Water
|-- Fire
|-- FloorTrap
|-- ArrowTrap
|-- Staircase
|-- Treasure
|-- ItemPickup
'-- LitBomb
```

The walls. The two wall types share the conceptual category “wall,” but they have very little in common at the code level. `IndestructibleWall` is just a coloured tile that blocks movement; it extends `Entity` directly. `DestructibleWall` has HP, takes damage when the player attacks it, and consumes itself when its HP reaches zero, so it extends `LivingEntity`. Any abstract `Wall` class above them would have to sit somewhere awkward (above `LivingEntity`? In parallel?), and it would have nothing useful inside it because the two implementations share no code. Forcing the abstraction would have created an empty parent class just because the two children share a name. I left them as unrelated siblings.

Ghoul and CaveDog have identical tick logic: two steps per turn, attack on adjacency, but different stats. A solution could be to refactor them into a `TwoStepEnemy` parent class. I chose not to. With only two enemies sharing the pattern, the parent class would exist only to hold one method. If a third two-step enemy is added later, the refactoring could be worthwhile.

Both decisions came from the same idea, but for different reasons. The wall case was about resisting an abstraction that the type system couldn’t support; the enemy case was about resisting an abstraction that the code base didn’t yet need.

2.2 Encapsulation

Each class controls its own state and exposes only what callers need. `Player` keeps its inventory in a private `ArrayList` and returns a defensive copy from `getInventory`; callers cannot mutate the list directly. Health is similarly encapsulated: external code never assigns to the field, it can only call `damage` or `heal`, both of which clamp at the boundaries (zero and max health). The invariant $0 \leq \text{health} \leq \text{maxHealth}$ is enforced by the class itself, not by the discipline of callers. The `consumed` flag on `Entity` works the same way. `isConsumed()` reads the flag, `consume()` sets it; there is no setter that takes a boolean and no way to un-consume. An entity’s lifecycle only moves in one direction, and the class enforces that. The advantage shows up in `Dungeon.cleanup`, which can safely sweep consumed entities at the end of every turn knowing they will never come back. Encapsulation also takes the form of what is not exposed. Entities do not hold references to `Game`. They communicate with the engine through small public methods on `Player` (`setWon`, `requestNextLevel`, `stageBomb`).

2.3 Polymorphism in practice and key design decisions

Each turn, the game loop has to decide what happens when the player walks into a tile. It uses polymorphism to do this without knowing the concrete entity type.

```
Entity targetEntity = currentDungeon().entityAt(targetX, targetY);
if (targetEntity != null) {
    String result = targetEntity.interactWith(player);
    boolean canEnter = !targetEntity.isOccupying();
    ...
}
```

Game does not know whether the entity at that tile is a wall, a water tile, a zombie, a piece of treasure, or a lit bomb. It calls `interactWith` and the right thing happens for each type. The only place that branches on entity type by symbol is `DungeonLoader`, which has to know what character maps to what class. Adding all four enemies, both wall types, both trap types, the bomb, the staircase, and the item pickup needed no changes to **Game** or **Dungeon**. The only `instanceof` checks in the engine live inside `LitBomb`'s explosion code, where the blast needs to identify living entities (to apply damage) and enemies by name (to format the casualty message). These checks are local to the explosion logic and do not affect any other part of the engine.

Player flags for cross-entity signaling. When an entity needs to tell **Game** that something happened, it sets a flag on the **Player** rather than calling into **Game** directly. `Treasure.interactWith` calls `player.setWon()`; `Staircase.interactWith` calls `player.requestNextLevel()`; `Bomb.use` stages a `LitBomb` on the player for **Game** to place into the dungeon. **Game** reads these flags after the interaction phase and acts on them. The advantage is that none of the entity classes needs a reference to **Game**, and **Game** never has to ask what kind of entity that was; it just checks the flags. By the end I was using this pattern in four places, and it had become the standard way for entities to communicate with **Game**.

interactWith returns a String. Every entity's interaction returns a status message, even if that message is empty. The reason is that each entity should decide what message it produces. Early in the project, **Game** was responsible for the line "Ouch! You bumped into something solid" whenever the player walked into a blocking entity. That worked for walls, but it meant **Game** was implicitly deciding what "bumping" feels like for every entity type. When I added enemies and the destructible wall, both of which want to narrate their own bump message, the line in **Game** started overriding them. The fix was to let the entity speak first and only fall back to the generic bump message if the entity returned an empty string. That small change made it possible for every new blocking entity to define its own narration without touching **Game**.

The consumed flag and end-of-turn cleanup. When an entity needs to be removed from the dungeon (a consumed potion, a sprung trap, a dead enemy), it does not directly remove itself from **Dungeon**'s entity list. Instead, it sets an internal `consumed` flag. At the end of the turn, `Dungeon.cleanup()` runs `entities.removeIf(Entity::isConsumed)` and sweeps them all out at once. This avoids two problems: an entity modifying the list it is being iterated on (which would throw a `ConcurrentModificationException`), and an entity vanishing mid-turn while another entity is mid-way through interacting with it. The cleanup is one line and runs once per tick.

Data-driven level format. Levels are described in plain text files, not built in code. `DungeonLoader` reads a text file and turns each character into the corresponding entity.

Bomb and LitBomb as two classes. The bomb item in the inventory and the lit bomb on the floor are different things with different lifecycles. The inventory representation does not need to tick or know about the dungeon; it just needs to know its stats and how to spawn its lit

form when used. The lit version lives in the dungeon, ticks every turn, and explodes. Squeezing both behaviours into one class would have meant adding `tick()` to the `Item` contract, which no other item needs. Two classes with one well-defined transformation point (`Bomb.use` stages a `LitBomb` that `Game` places into the dungeon) keep each class doing one thing.

2.4 Testing strategy

The test suite uses JUnit 5 and is structured around the spec's emphasis on black-box testing: the tests verify what each class does through its public methods, not how it does it internally. Across seven test files there are 43 tests total, covering the behaviours that matter most for the spec and for the design decisions I made along the way.

The tests fall into roughly three categories. The first is invariants that should hold no matter how the rest of the engine treats the class. `Player.heal` should never exceed `maxHealth`; `Player.damage` should never push health below zero; `Player.getInventory` should return a defensive copy that callers cannot use to mutate the underlying list. These tests pin down properties of the class that the entire engine depends on. If any of them fails in the future, something fundamental has broken.

The second category is design decisions. The combat exchange in `Enemy.interactWith` is supposed to let the player strike first; a test confirms a lethal first blow kills the enemy before it counter-attacks, so the player takes no damage. The staircase deliberately does not consume itself when stepped on (in case backtracking is ever added); a test confirms it stays in the dungeon after firing, the tests catch unintended changes to that behaviour.

The third category is contract tests on classes that read external input. `DungeonLoader` has five tests that verify it loads a valid file correctly, throws on unknown characters, throws when the player start is missing, throws on an empty file, and throws on a missing file.

3 Limitations and future work

Enemy pathfinding is greedy. The shared `stepToward` helper picks the axis with the larger remaining distance and steps along it; if that axis is blocked, it tries the other. This works well in open spaces and around isolated obstacles, but it can stall against concave shapes such as L-shaped walls, where the correct path requires temporarily moving *away* from the player. A proper breadth-first search would solve this by exploring every reachable tile in expanding rings until it finds the player, guaranteeing the shortest path. An A* search would be the more efficient option for larger dungeons, since it uses a heuristic (Manhattan distance to the player) to prioritise tiles that look closer to the goal, exploring fewer dead ends than BFS. Both approaches would require a visited set, a priority queue or FIFO queue, and per-tick allocation for each enemy. For a small dungeon with at most a handful of enemies and short distances to the player, the greedy approach is good enough, and when this happens the enemy just stops moving; it does not crash or do anything strange.

No save/load between sessions. The game supports restart, which reloads the level files from disk and resets all state, but it does not save progress to disk. A player who closes the program loses their position, HP, and inventory. Implementing this would mean serialising the player's state and the current level index to a file, then restoring on startup. The architecture supports it cleanly because the levels are already file-based; only the runtime state would need persistence.

Level design is minimal and hand-crafted. The architecture supports any number of levels and any layout, but the game only has two hand-designed maps. A real roguelike would have more levels, an explicit difficulty curve where enemy stats and trap density scale upward, and item placement chosen to match each level's intended challenges. Every playthrough sees the same dungeon. The data-driven loader could in principle be fed by a generator instead of a static file. A further limitation is the level format itself: one entity per character, with stats hardcoded in the loader. A designer cannot place a “weak zombie” next to a “strong zombie”, because every Z produces an identical `Zombie` instance. A bigger file format with per-instance parameters would unlock more interesting level design.

Input requires pressing Enter after each command. The game reads input one line at a time through a `Scanner`, so every move requires a keystroke plus the Enter key. Classic roguelike games use single-keystroke input where each press is processed immediately. I used only Java's standard library that does not expose the terminal mode that single command requires. Achieving a single command would mean using a third-party library such as `JLine`. This was a deliberate trade-off: it works identically on Windows, macOS, and Linux without external dependencies, at the cost of an Enter press per turn.

4 Reflection

The decision that surprised me most was building a minimal playable version first, before any of the spec’s specific features. A player on a grid with walls and one item made me settle the **Entity** contract, the rendering layer, the input loop, and the bump-interaction model. After that, almost every new feature was a new class extending **Entity** plus a line in the loader.

The spec said about the bomb that it “damages surrounding entities in some radius after some amount of moves,” which leaves open where the explosion happens and whether the bomb stays in inventory. I tried one interpretation that did not work before settling on the placed-bomb approach: lighting the bomb turns it from an **Item** into a **LitBomb** entity placed in the dungeon. **Bomb.use** stages a **LitBomb**, **Game** adds it after the action phase, and after that it is just another ticking entity. Putting both behaviors in one class would have required adding **tick** to **Item** for a single use case.

A pattern I did not plan but used three times was setting flags on **Player** to signal events back to **Game**. It started with **hasWon** when I replaced an **instanceof Treasure** check in the game loop. Later, **wantsNextLevel** for the staircase and **pendingBomb** for the bomb followed the same shape.

I was most uncertain about the testing part, but when I got it up and running it was not that bad. The first test took the longest by far; most of the time went into JUnit setup rather than the test itself. After that the pattern became obvious: construct the object, call the method and check the result. The classes I had built were small and I think it is why the tests were easy to write; when a class does one thing, testing it is also one thing.